



Universidade do Minho
Licenciatura em Ciências da Computação

Unidade Curricular de Computação Gráfica

Ano Letivo de 2023/2024

Normais e Coordenadas de Textura – Fase 4

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Maio, 2024

CG

Data de Recepção	
Responsável	
Avaliação	
Observações	

Normais e Coordenadas de Textura – Fase 4

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Maio, 2024

Introdução

O presente relatório é referente à quarta fase de um projeto desenvolvido no âmbito da unidade curricular de Computação Gráfica, tendo como propósito obter uma representação dinâmica e mais realista das cenas, através da adição de texturas e iluminação.

Afim da possibilidade da escrita no ficheiro *.3d* das coordenadas de textura e das normais de cada vértice foi necessário alterar cada primitiva no *generator*.

No *engine* foram feitas alterações para ser possível a leitura dos novos ficheiros *.3d* e para suportar a iluminação e aplicação de texturas às primitivas.

Ao longo deste relatório irão ser explicadas, de forma detalhada, todas as decisões tomadas.

Índice

Introdução	i
Índice	ii
Índice de Figuras	iii
1. <i>Estruturas de dados</i>	1
2. Generator	3
2.1. Ficheiro <i>.3d</i>	3
2.2. Normais e coordenadas de textura	3
2.2.1 Plano	3
2.2.2 Caixa (<i>Box</i>)	5
2.2.3 Cone	5
2.2.4 Esfera	7
2.2.5 Ring	8
2.2.6 Superfícies de <i>Bezier</i>	8
3. <i>Engine</i>	10
3.1. Iluminação	10
3.2. Texturas e Normais	11
4. Sistema Solar	13
5. Resultados	14
5.1. Capturas referentes aos testes fornecidos	14
5.2. Capturas referentes a testes extra	15
6. Conclusão	17

Índice de Figuras

Figura 1 – Plano XZ	4
Figura 2 – Textura Plano XZ	4
Figura 3 – Plano YZ	4
Figura 4 – Textura Plano YZ	4
Figura 5 – Plano XY	4
Figura 6 – Textura Plano XY	4
Figura 7 – Coordenadas X e Z	5
Figura 8 – Coordenada Y	5
Figura 9 – Ponto Médio e Raio	6
Figura 10 – Raio correspondente a cada <i>stack</i>	7
Figura 11 – Textura Cone	7
Figura 12 – Textura Esfera	7
Figura 13 – Textura Ring	8
Figura 14 – Textura Bezier	9
Figura 15 – Demonstração Sistema Solar (instante 27)	13
Figura 16 – Demonstração Sistema Solar (instante 38)	13
Figura 17 – Demonstração Sistema Solar (instante 48)	13
Figura 18 – test_4_1	14
Figura 19 – test_4_2	14
Figura 20 – test_4_3	14
Figura 21 – test_4_4	14
Figura 22 – test_4_5	15
Figura 23 – test_4_6	15
Figura 24 – test_4_cone	15
Figura 25 – teste_4_ring	15
Figura 26 – test_4_bezier	16
Figura 27 – test_4_earth	16
Figura 28 - solar_system_static_no_texture	16

1. *Estruturas de dados*

Com vista a suportar as funcionalidades propostas nesta fase do projeto foram atualizadas as estruturas de dados.

A estrutura *Config* foi alterada para conseguir armazenar as informações relativas à iluminação através do vetor *lights*; para além disso, a informação dos *models* passa a ser guardada recorrendo à *struct ModelInfo* que permite guardar os dados dos vértices, das normais e das coordenadas de textura.

Também a *struct Group* foi modificada para guardar informações que permitem personalizar os modelos (*ModelSettings*).

```
struct ModelSettings
{
    string modelFile;
    string textureFile;
    Color color;
};

struct ModelInfo
{
    vector<Point> vertices;
    vector<Point> normals;
    vector<TexCoord> texCoords;
};

struct Group
{
    vector<Transform> transforms;
    vector<ModelSettings> models;
    vector<Group> groups;
};
```

```
struct Config
{
    int window[2];
    float position[3];
    float lookAt[3];
    float up[3];
    float projection[3];
    vector<Light> lights;
    map<string, ModellInfo> models;
    Group group;
};
```

2. Generator

2.1. Ficheiro .3d

Até esta fase apenas era imperativo guardar nos ficheiros .3d as coordenadas dos pontos necessárias para desenhar uma primitiva; portanto, e de maneira a serem guardadas as coordenadas dos vetores normais e as coordenadas de textura, alterou-se os ficheiros .3d que, passam então a ter a seguinte estrutura:

```
-1 0 -1 0 1 0 0 1
```

```
-0.333333 0 -0.333333 0 1 0 0.333333 0.666667
```

```
-0.333333 0 -1 0 1 0 0.333333 1
```

```
-0.333333 0 -0.333333 0 1 0 0.333333 0.666667
```

```
-1 0 -1 0 1 0 0 1
```

Coordenadas dos pontos

Coordenadas dos vetores normais

Coordenadas de textura

2.2. Normais e coordenadas de textura

2.2.1 Plano

A normal de qualquer ponto pertencente ao plano XY , se seguir a orientação do eixo z é $(0, 0, 1)$, se tiver a orientação inversa é $(0, 0, -1)$. Já a normal de qualquer ponto pertencente ao plano XZ , se seguir a orientação do eixo y é $(0, 1, 0)$, se tiver a orientação inversa é $(0, -1, 0)$. Por sua vez, a normal de qualquer ponto pertencente ao plano YZ , se seguir a orientação do eixo x é $(1, 0, 0)$, se tiver a orientação inversa é $(-1, 0, 0)$.

As coordenadas de textura de cada plano são obtidas através do mapeamento entre os vértices das primitivas e os seus equivalentes na textura, tal como está representado nas seguintes figuras:

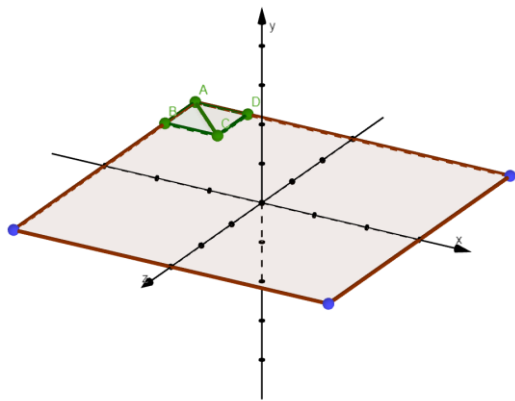


Figura 1 - Plano XZ

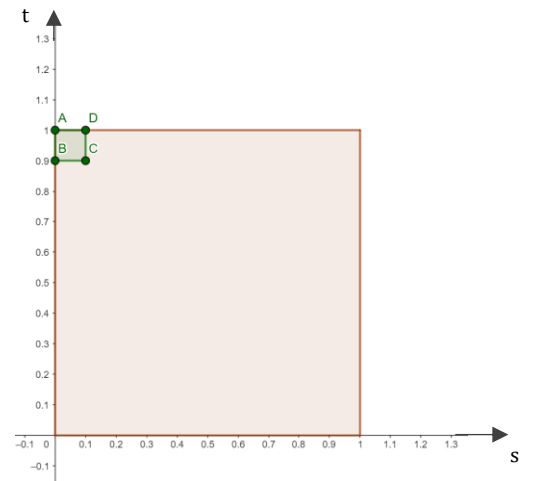


Figura 2 - Textura Plano XZ

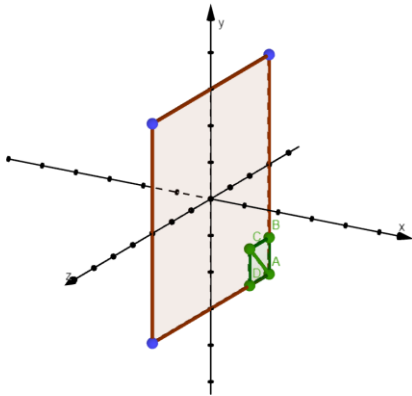


Figura 3 - Plano YZ

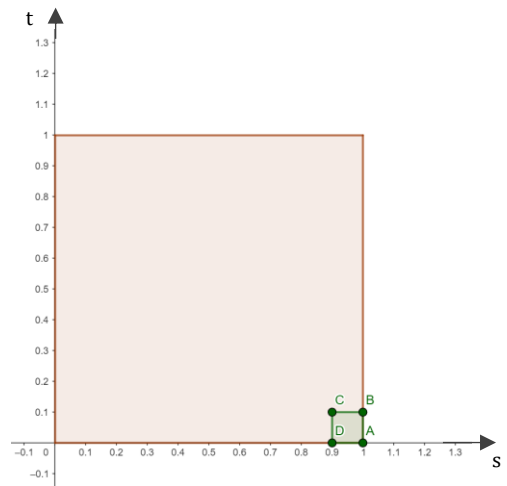


Figura 4 - Textura Plano YZ

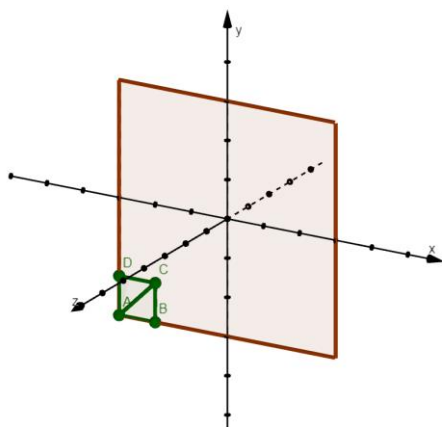


Figura 5 - Plano XY

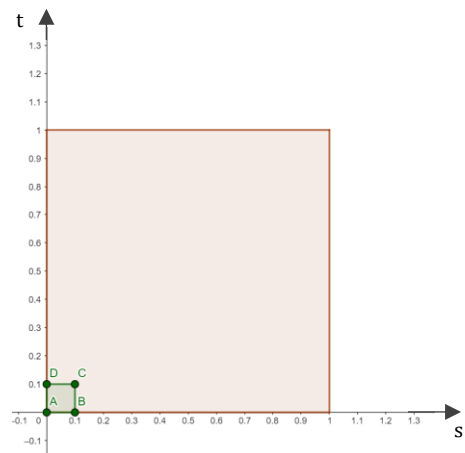


Figura 6 - Textura Plano XY

2.2.2 Caixa (Box)

Visto que a caixa é um conjunto de seis planos, as coordenadas dos vetores normais e as coordenadas de textura são geradas no processo de criação de cada um dos planos, tal como descrito anteriormente.

2.2.3 Cone

O cálculo das normais dos pontos do cone é feito em duas fases, primeiro calcula-se as normais dos pontos da base e em seguida dos pontos das faces laterais.

As coordenadas das normais dos pontos da base são $(0, -1, 0)$. Para o cálculo das normais dos pontos das faces laterais, obtêm-se as coordenadas X e Z da normal de cada ponto através das funções trigonométricas (*Figura 7*), considerando o raio da *stack* que estamos a iterar. A coordenada Y da normal é calculada recorrendo à semelhança de triângulos e ao ângulo β_2 tal como representado na *Figura 8*.

Após os cálculos das coordenadas, é feita a normalização do vetor normal.

```
float beta = atan((float)radius / (float)height);  
float ny = sin(beta);
```

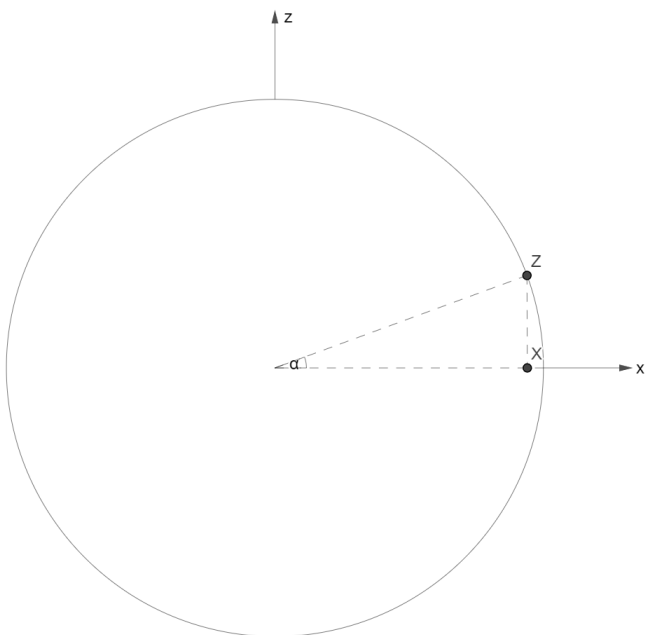


Figura 7 - Coordenadas X e Z

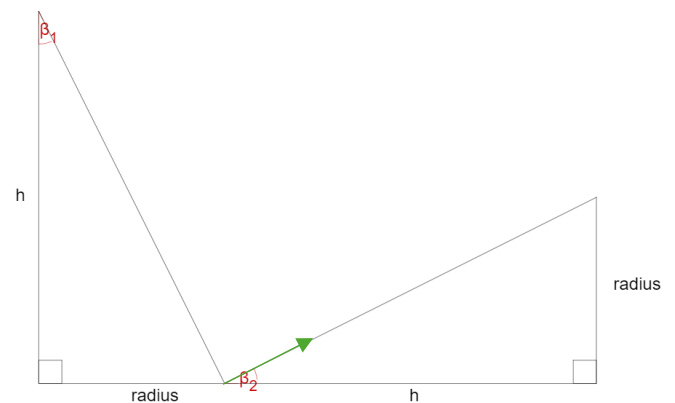


Figura 8 - Coordenada Y

Respetivamente ao cálculo das coordenadas de textura, para a base do cone determinou-se o ponto médio e o raio (*Figura 9*). Para o mapeamento da base do cone para a textura, foram utilizadas as funções trigonométricas, sendo que a variável i é referente à *slice* em que se está a iterar.

$$Tx = PM + raio * \cos(\alpha * i)$$

$$Ty = PM + raio * \sin(\alpha * i)$$

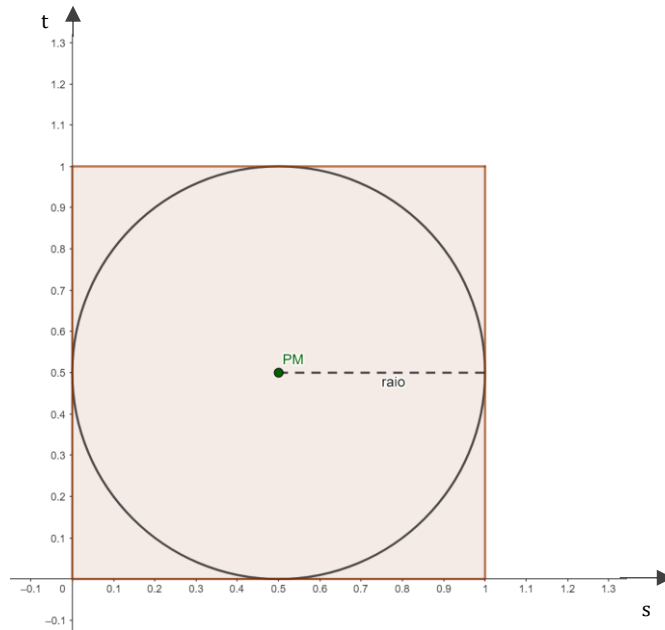


Figura 9 - Ponto Médio e Raio

Para o cálculo das coordenadas de textura das faces laterais, é seguido o mesmo raciocínio explicado anteriormente, sendo que, agora é necessário ter em conta as *slices* e as *stacks* em que se está a iterar. Como é mostrado na *Figura 10* e *Figura 11*, é necessário calcular frequentemente o raio interior e raio exterior das circunferências que dizem respeito a cada *stack*.

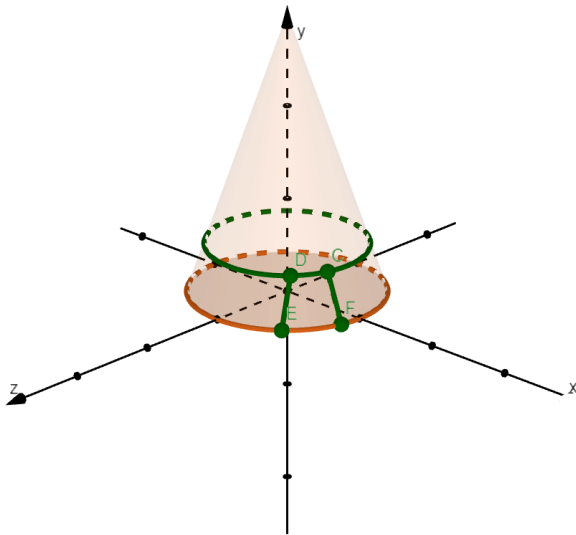


Figura 10 – Raio correspondente a cada *stack*

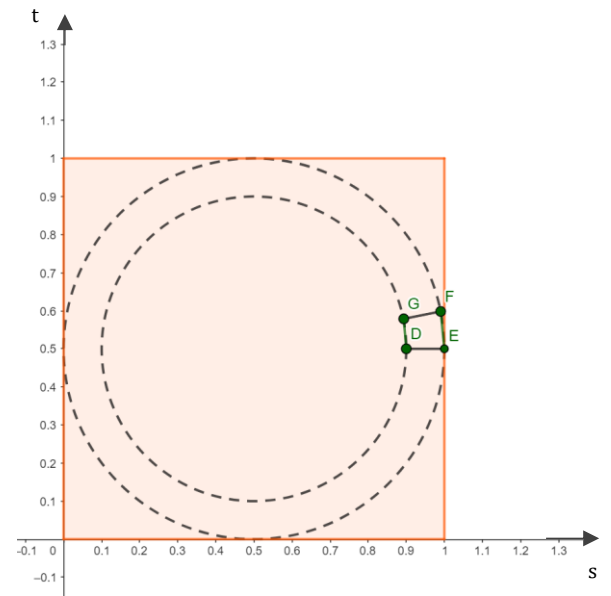


Figura 11 – Textura Cone

2.2.4 Esfera

No caso da esfera, os cálculos das coordenadas dos vetores normais serão iguais ao vetor normalizado que vai do centro da esfera até ao ponto da superfície. Desta forma apenas é necessário dividir o valor das coordenadas dos pontos pelo raio.

Para fazer o mapeamento entre a primitiva e a textura, foi feita a divisão da altura da textura pelo número de *stacks* e, a divisão da largura pelo número de *slices*. Cada quadrado formado por esta divisão será mapeado para a correspondente secção da esfera (secção formada pela intersecção das *slices* e das *stacks*).

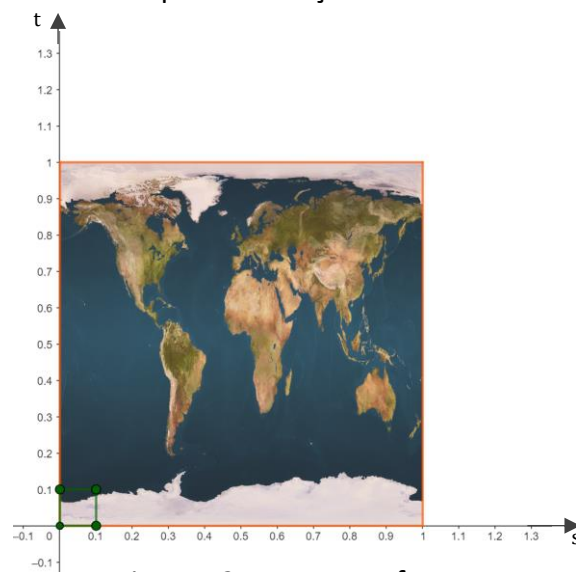


Figura 12 - Textura Esfera

2.2.5 Ring

O vetor normal de cada ponto pertencente ao ring será $(0, 0, 1)$ na parte superior e $(0, 0, -1)$ na parte inferior.

Tal como se observa na *Figura 13*, é feita a divisão da textura pelo número de *slices* e para cada vértice, os vértices superiores têm coordenada de textura $Y = 1$,

$X = i * (\Delta slices)$ e os vértices inferiores têm coordenada de textura $Y = 0$,

$X = i * (\Delta slices)$, sendo que a variável i diz respeito à *slice* em que se está a iterar e

Δ o intervalo entre slices consecutivas.

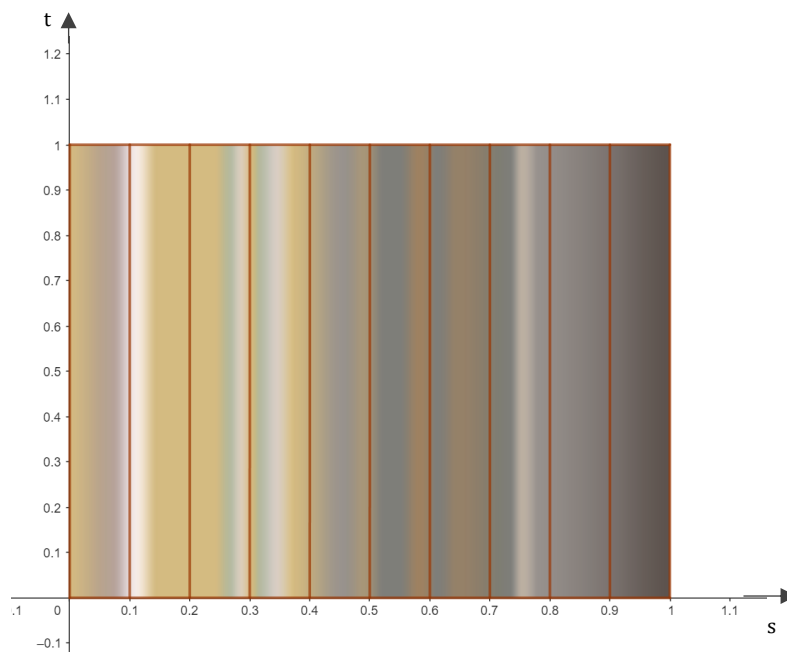


Figura 13 - Textura Ring

2.2.6 Superfícies de *Bezier*

Para calcular as coordenadas dos vetores normais das superfícies de Bezier recorreu-se às seguintes fórmulas:

$$M = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = M^T$$

$$P(u, v) = [u^3 \ u^2 \ u \ 1] M \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} M^T \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

$$\frac{\partial(P(u,v))}{\partial u} = [3u^2 \ 2u \ 1 \ 0] M \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} M^T \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

$$\frac{\partial(P(u,v))}{\partial v} = [u^3 \ u^2 \ u \ 1] M \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} M^T \begin{bmatrix} 3v^2 \\ 2v \\ 1 \\ 0 \end{bmatrix}$$

O vetor normal para cada ponto da superfície é obtido através da normalização do resultado do produto externo entre os vetores tangentes obtidos.

No caso do cálculo das coordenadas de textura, para cada patch é aplicada uma textura e, a fim de fazer o mapeamento entre cada textura e cada patch recorre-se a u e v das fórmulas anteriores. Obtêm-se, portanto, as coordenadas de textura por $(1 - v, 1 - u)$, de acordo com a figura seguinte:

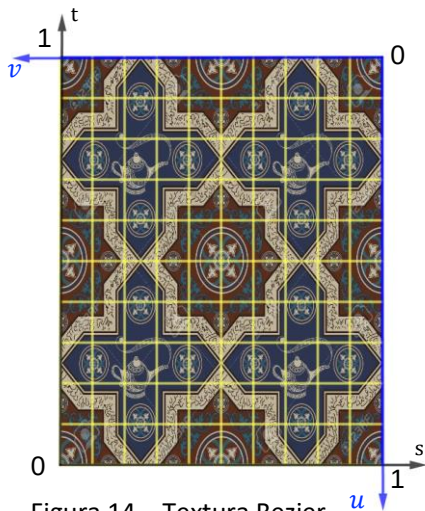


Figura 14 – Textura Bezier

3. Engine

3.1. Iluminação

Para suportar as funcionalidades de iluminação foram criadas as seguintes estruturas:

- *struct Light*

```
struct Light
{
    int type;
    float pos[4] = {0.0f, 0.0f, 0.0f, 1.0f};
    float dir[4] = {0.0f, 0.0f, 0.0f, 0.0f};
    float cuttOff;
```

No campo *type* são aceites os valores 0, 1 e 2, que dizem respeito aos tipos de luz *point*, *directional* e *spotlight*, respetivamente. Os campos *pos* e *dir* contêm informação sobre a posição e a direção da fonte de luz. Já o campo *cuttOff*, que especifica o ângulo máximo de propagação de uma fonte de luz, só é utilizado se a fonte de luz for um *spotlight*.

- *struct Color*

```
struct Color
{
    float diffuse[3] = {200.0f, 200.0f, 200.0f};
    float ambient[3] = {50.0f, 50.0f, 50.0f};
    float specular[3] = {0.0f, 0.0f, 0.0f};
    float emissive[3] = {0.0f, 0.0f, 0.0f};
    float shininess = 0.0f;
```

Nos campos *diffuse*, *ambient*, *specular*, *emissive* e *shininess*, são guardados os valores das componentes difusa, ambiente, especular, emissiva e o coeficiente de especularidade das cores dos modelos, respetivamente. *Shininess* aceita valores entre [0, 128] enquanto cada um dos outros campos aceita valores RGBA (cada componente aceita valores entre [0, 1]).

Para implementar a funcionalidade de iluminação, foi necessário atualizar a aplicação *Engine*.

Inicialmente, recorrendo às funções de *OpenGL* *glEnable(GL_LIGHTING)* e *glEnable(GL_LIGHTi)* (com $i \in [0, 7]$), é ligado o “quadro” e cada luz, individualmente.

De seguida, na função *renderScene* invoca-se a função *activateLights*, que permite posicionar e definir cada uma das fontes de luz. Para tal, faz-se uso da função *glLightfv* e do parâmetro *GL_POSITION*. No caso da luz ser do tipo *spotlight*, além do passo mencionado anteriormente, faz-se também uso da função *glLightf* e dos parâmetros *GL_SPOT_DIRECTION*, *GL_SPOT_CUTOFF* e *GL_SPOT_EXPONENT*.

Já para colorir os modelos, na função *drawModelsVBO*, para cada modelo definiram-se as componentes difusa, ambiente, especular e emissiva recorrendo à função *glMaterialfv* e aos parâmetros *GL_DIFFUSE*, *GL_AMBIENT*, *GL_SPECULAR*, *GL_EMISSION*, respetivamente. Para definir o *Shininess*, é usada a função *glMaterialf* e o parâmetro *GL_SHININESS*.

3.2. Texturas e Normais

Para ler e aplicar as coordenadas de textura e as coordenadas das normais dos ficheiros *.3d* dos modelos, fez-se um processo semelhante ao que foi feito para implementar *VBO's*.

Inicialmente, alocou-se memória e foram criados buffers para os vetores normais e para as coordenadas de textura. Além disso, também foi alocada memória para um array de *ID's* de texturas.

```
nBuffers = (GLuint *)malloc(sizeof(GLuint) * nModels);
texBuffers = (GLuint *)malloc(sizeof(GLuint) * nModels);
// Array of textures IDs
texIDs = (GLuint *)malloc(sizeof(GLuint) * nModels);

// Create buffers
glGenBuffers(nModels, nBuffers);
glGenBuffers(nModels, texBuffers);
```

De seguida, é invocada a função *modelsToBuffers*, que vai percorrer todos os modelos e carregar os dados necessários (vértices, vetores normais e coordenadas de textura), para os respetivos buffers. É também nesta função que, no caso de haver um ficheiro de textura associado ao modelo, é feito *load* da textura, através da função *loadTexture*,

e guarda-se o respetivo *ID* no array *texIDs*. Posteriormente, na fase de desenho dos VBO's (função *drawModelsVBO*), no caso de haver uma textura associada ao modelo, faz-se *bind* da textura, consultando o array de *ID*'s de texturas.

```
glBindTexture(GL_TEXTURE_2D, texIDs[*index]);
```

4. Sistema Solar

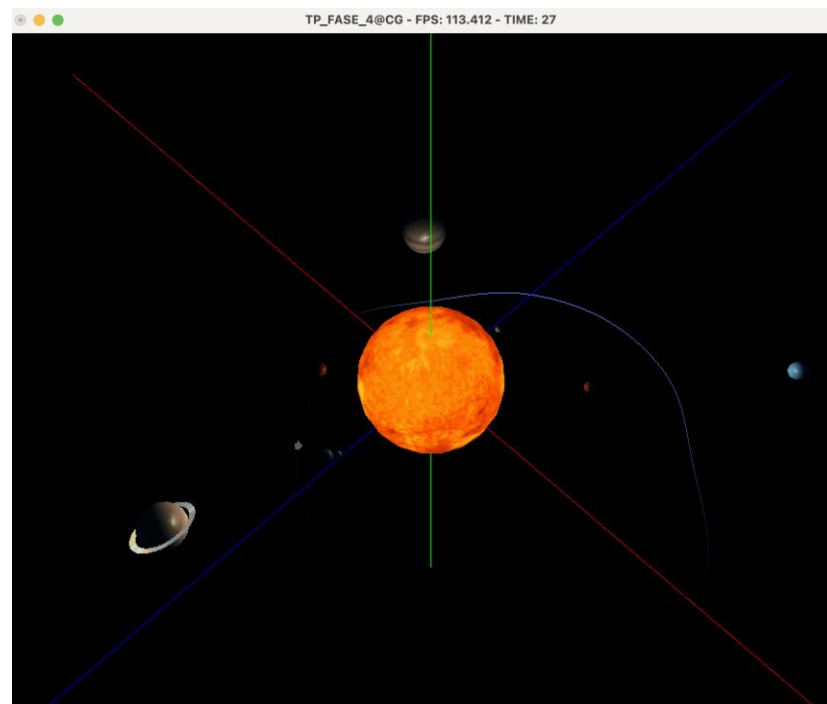


Figura 15 - Demonstração Sistema Solar (instante 27)

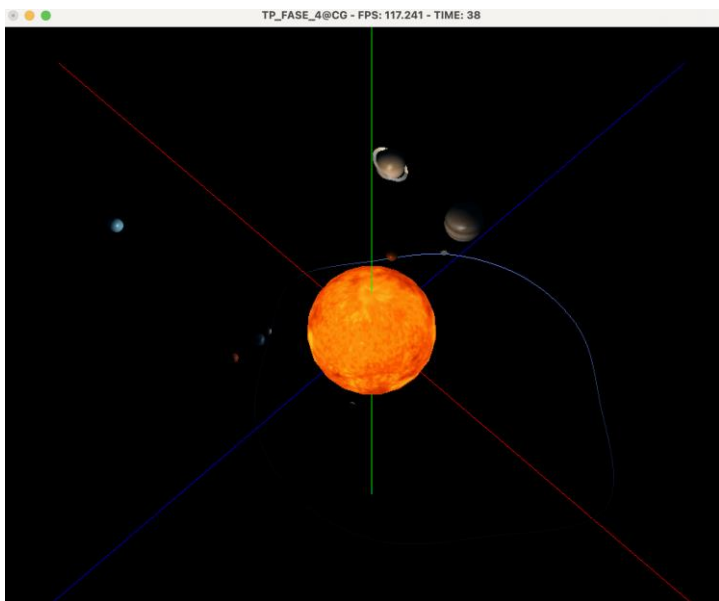


Figura 16 - Demonstração Sistema Solar (instante 38)

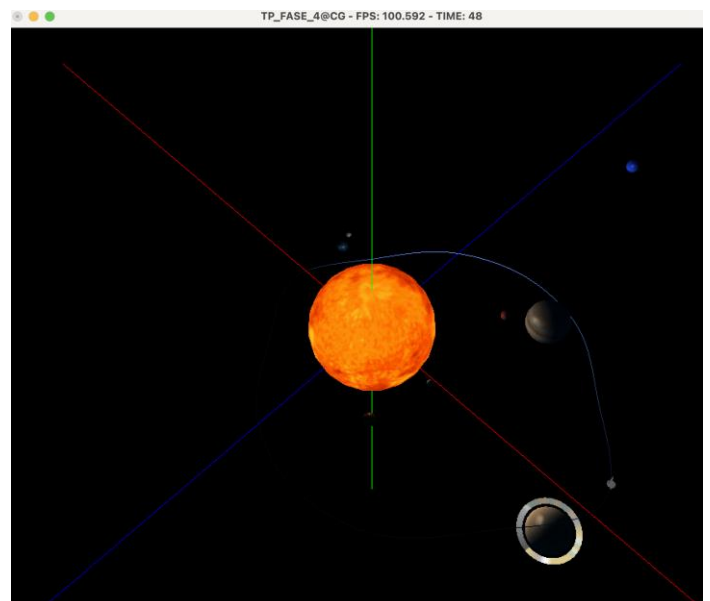


Figura 17 - Demonstração Sistema Solar (instante 48)

5. Resultados

5.1. Capturas referentes aos testes fornecidos

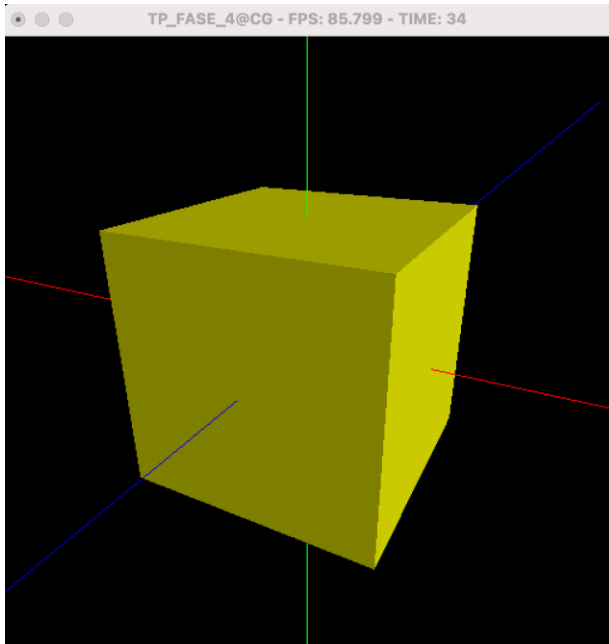


Figura 18 - test_4_1

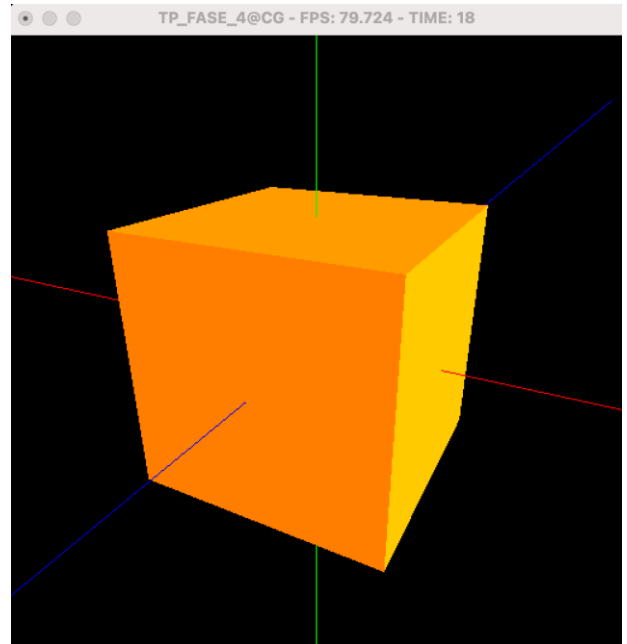


Figura 19 - test_4_2

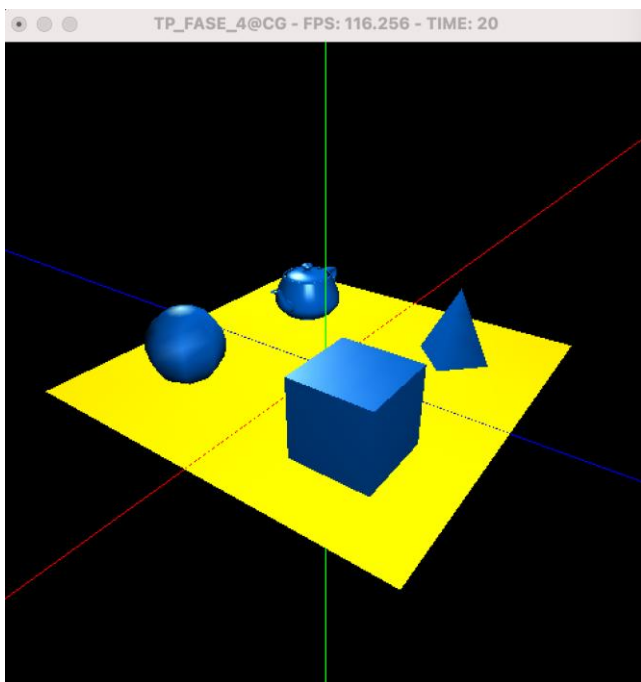


Figura 20 - test_4_3

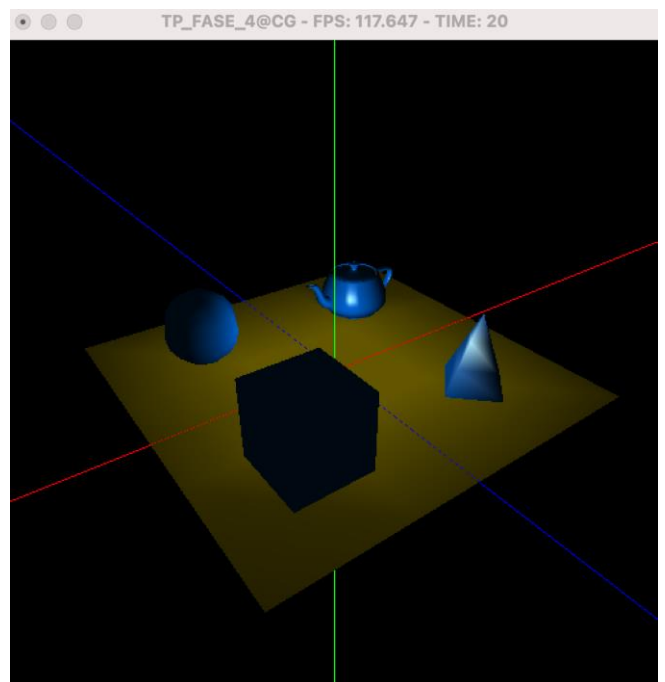


Figura 21 - test_4_4

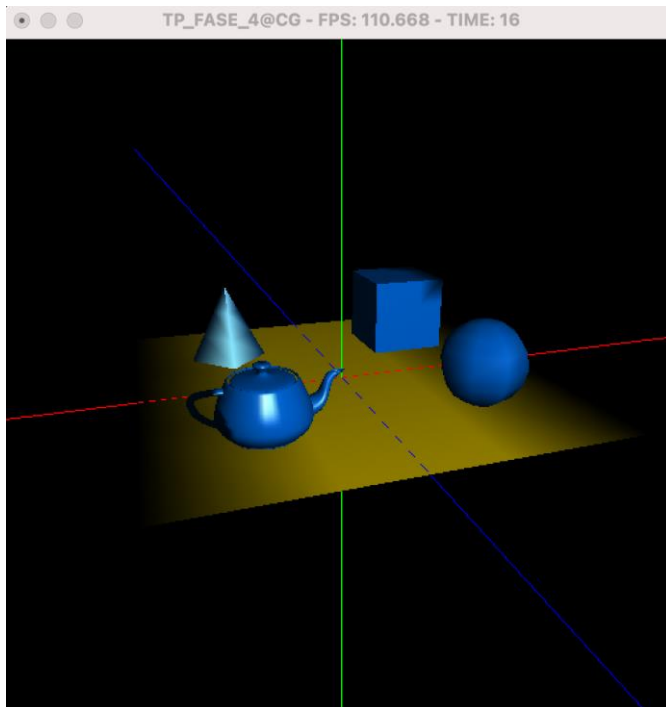


Figura 22 - test_4_5

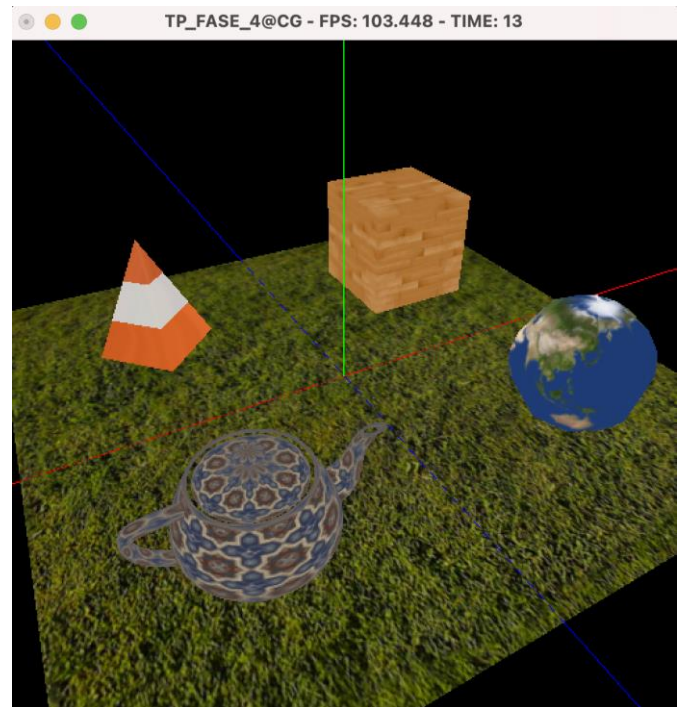


Figura 23 - test_4_6

5.2. Capturas referentes a testes extra

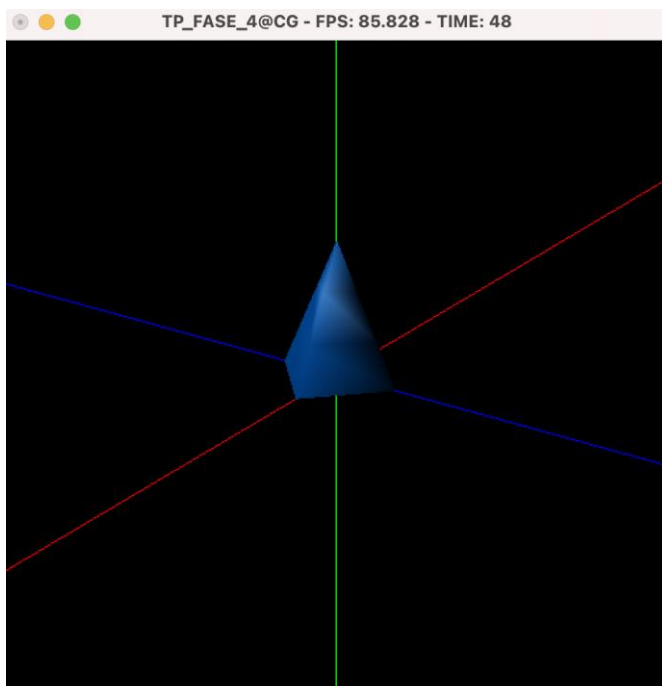


Figura 24 - test_4_cone

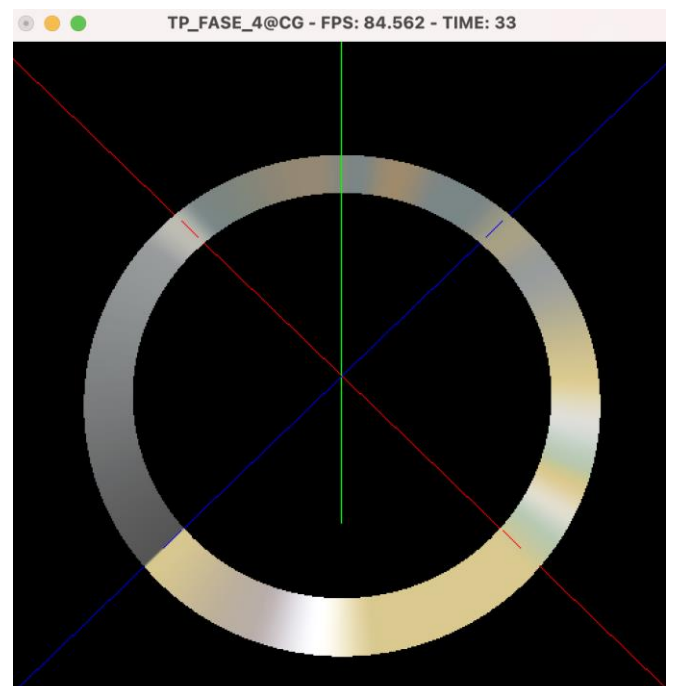


Figura 25 - test_4_ring

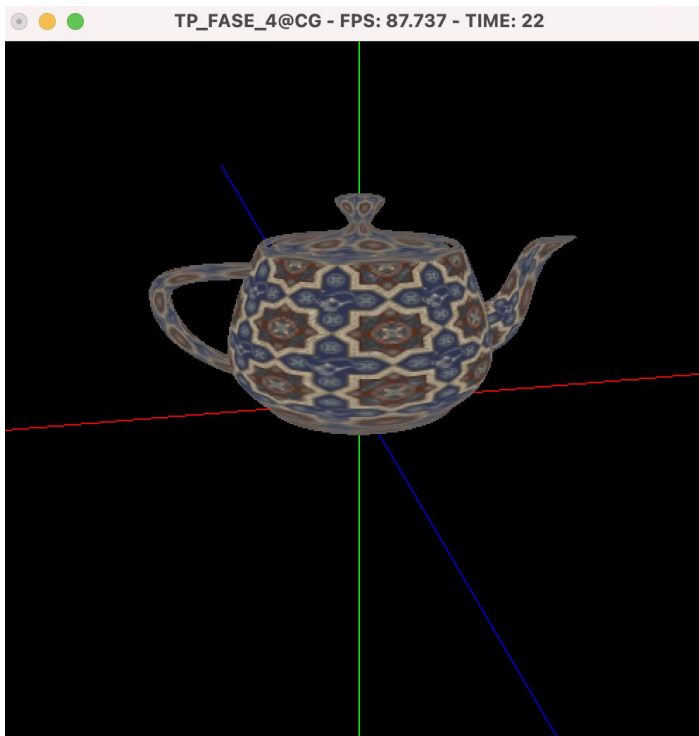


Figura 26 - test_4_bezier

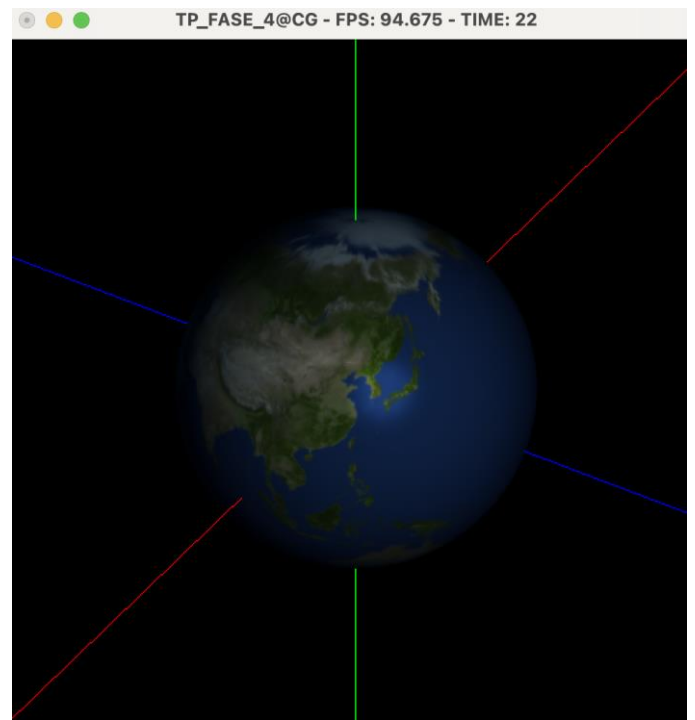


Figura 27 - test_4_earth

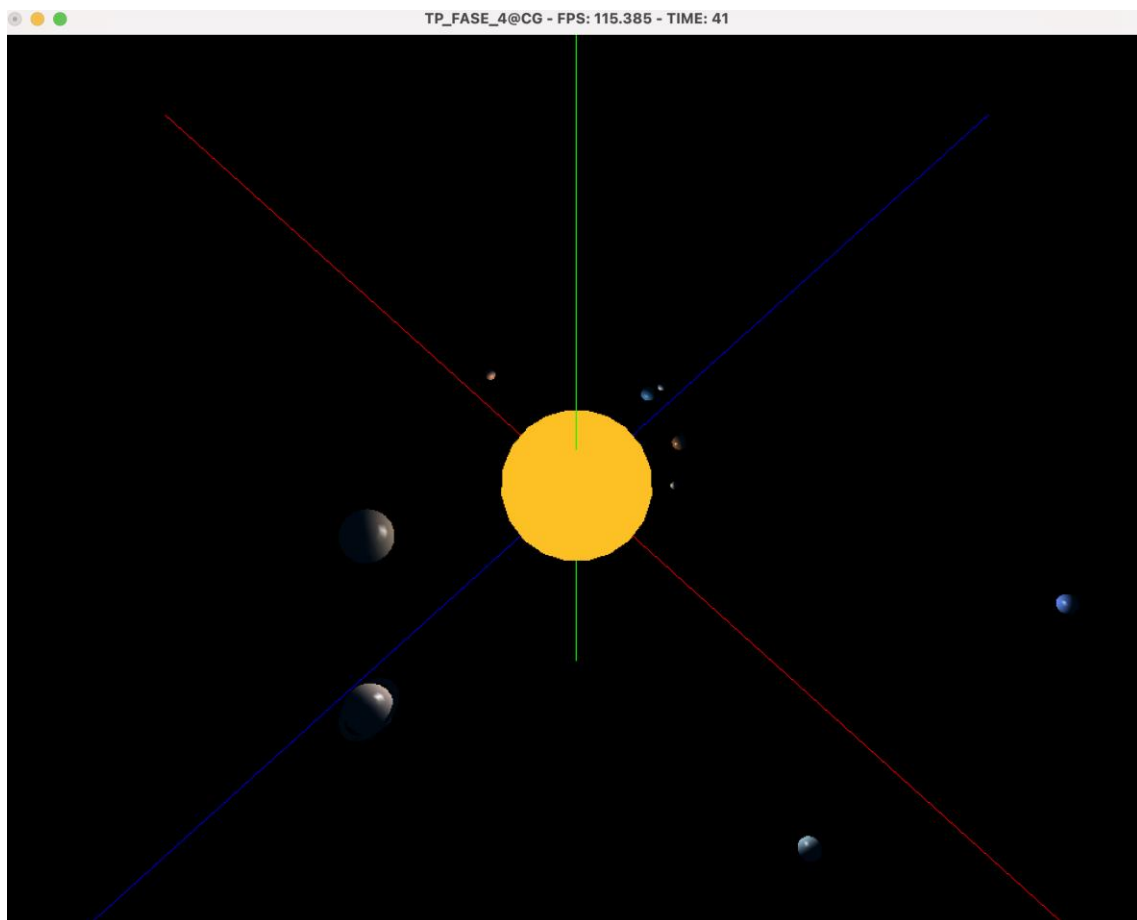


Figura 28 - solar_system_static_no_texture

6. Conclusão

Esta quarta fase do projeto de Computação Gráfica mostrou avanços significativos, apontando a uma representação mais realista do sistema solar através da implementação de texturas e luzes.

Assim, o *Generator* é capaz de gerar as normais e as coordenadas de textura para cada vértice dos vários modelos e, o *Engine* tem a capacidade de ativar as funcionalidades de iluminação e textura, assim como ler e aplicar as normais e coordenadas de textura dos *model files*.

Em suma, considera-se que os requisitos propostos foram cumpridos e as dificuldades sentidas foram superadas.